Otomatik Zafiyet Tarayıcı

Yazılım Temelli Proje Geliştirme Dersi - Ön Sunum

Ekip Üyeleri: Mücahit Haşim İPEK (1230606809), Necat Akyüz (1220606005)
Ders Sorumlusu: Prof. Dr. Ercan BULUŞ
Tarih: Ekim 2026

Problem ve Motivasyon

- Web uygulamalarına yönelik saldırılar sürekli artıyor.
- Manuel güvenlik testi yavaş ve pahalı.
- Küçük işletmeler ve geliştiriciler için basit, otomatik bir araç ihtiyacı var.
- Amaç: zafiyetleri saldırgandan önce bulup kapatmak.

Zafiyet Nedir?

- Tasarım, kodlama veya yapılandırma hatasından doğan güvenlik açığı.
- Temel neden: kullanıcı girdisine güvenmek ve hatalı yapılandırma.
- Basitten karmaşığa: bilgi ifşası, dizin listeleme, SQL Injection, XSS, CSRF, Command Injection.
- Referans: OWASP Top 10, PortSwigger Web Security Academy.

Proje Amacı ve Kapsamı

- Hedef web uygulamasını otomatik tarayıp zafiyetleri tespit etmek.
- Bulguları risk seviyesiyle raporlamak ve düzeltme önerisi sunmak.
- Kapsam: SQL Injection, XSS, açık port, güvenlik başlıkları, dizin listeleme, CVE eşleştirme.
- Yaklaşım: aşamalı geliştirme, yapabildiğimiz yere kadar.

Sistem Mimarisi

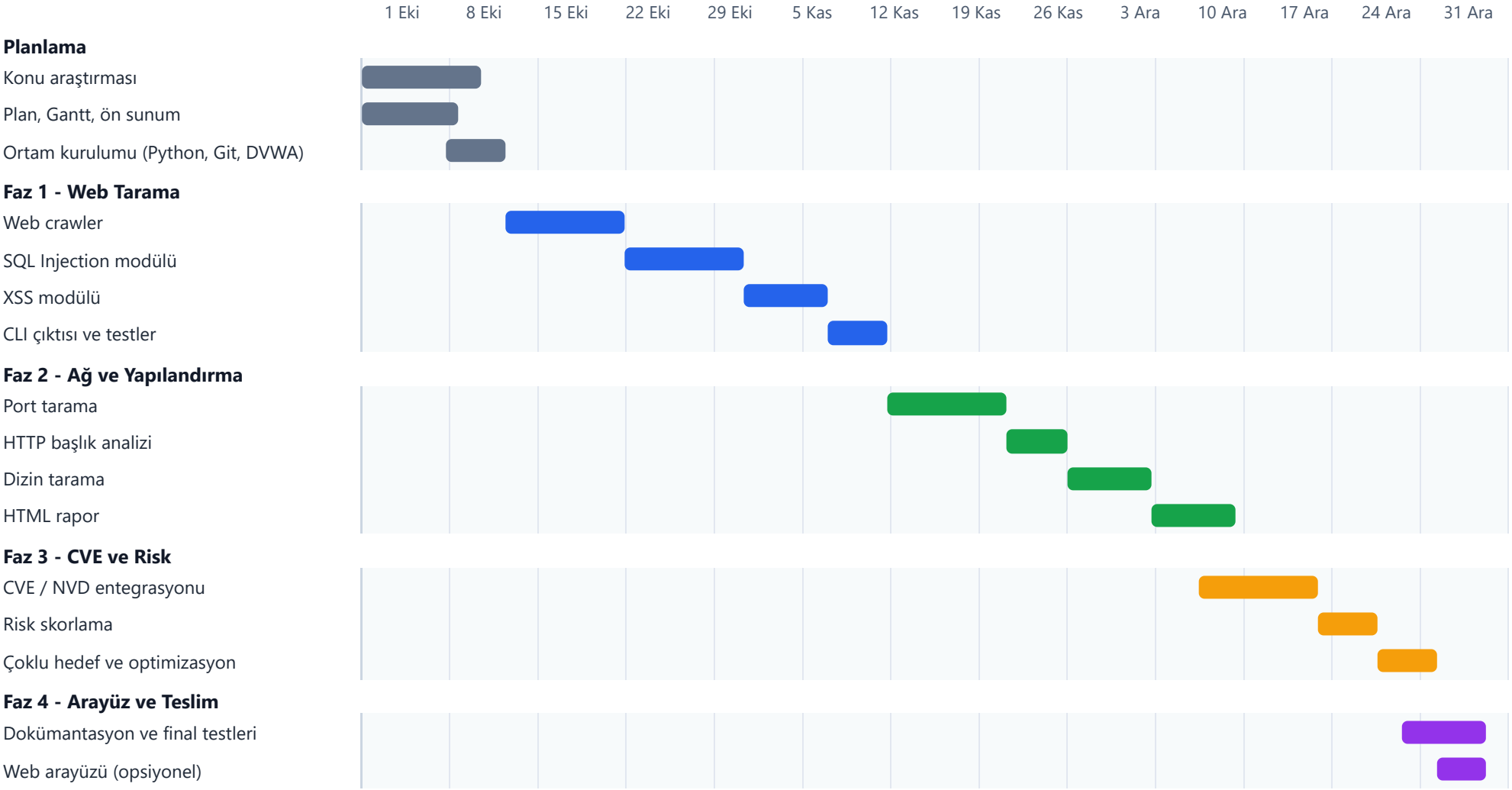
```
Hedef URL -> Crawler -> Tarama Modülleri -> Risk Skorlama -> Rapor
                          |- SQL Injection
                          |- XSS
                          |- Port / Başlık / Dizin
                          |- CVE kontrolü
```

- Modüler yapı: her zafiyet türü ayrı bir modül.
- Her bulgu: tür, URL, risk seviyesi, kanıt ve düzeltme önerisi.

Teknolojiler

Alan	Araç
Dil	Python
HTTP / Parsing	requests, BeautifulSoup
Ağ	socket / python-nmap
CVE verisi	NVD API
Rapor	Jinja2 (HTML)
Arayüz (opsiyonel)	Flask

Geliştirme Planı (Gantt) - 1 Ekim 2026 / 4 Ocak 2027



Etik ve Yasal Çerçeve

- Araç yalnızca izinli sistemlerde kullanılır.
- Test ortamı: kendi makinemizde DVWA ve OWASP Juice Shop.
- Gerçek kullanım: kapsamı belirlenmiş bug bounty programları (HackerOne, Bugcrowd).

> İzinsiz tarama suçtur. Araç, kullanıcıdan hedef sahipliği onayı alacaktır.

Beklenen Çıktılar ve Riskler

- Çıktılar: çalışan CLI aracı, HTML rapor, dokümantasyon, test sonuçları.
- Riskler: zaman yetersizliği, yanlış pozitifler, API limitleri.
- Önlemler: aşamalı geliştirme, bilinen zafiyetli uygulamalarla doğrulama.
- Zaman yetmezse arayüz ve çoklu hedef desteği çıkarılır; çekirdek hedef Faz 1 ve Faz 2.

Teşekkürler

Sorular ve geri bildirimler